Fine-Tuning and Testing of Grasping Tasks Based on $\pi_{0.5}$ Foundation Model for SO101 6-DOF Robot Arm

Yu-Fei Liu[†a]

[a]*School of Science and Engineering, The Chinese University of Hong Kong, Shenzhen*

Abstract

This project adapts the $\pi_{0.5}$ vision-language-action foundation model to two real-world grasping tasks on the SO101 6-DOF robot arm: picking a white milk box and pulling a tissue out of a tissue pack. The system combines leader-follower teleoperation, LeRobot-format demonstration datasets, Isaac Sim 5.1 trajectory replay with domain-randomized rendering, and OpenPI post-training with AdamW optimization. We collected 100 teleoperated trajectories in total, constructed paired front-camera and hand-camera observations, and fine-tuned the action model on an 8-card A800 server. The resulting pipeline provides a practical route from real demonstration collection to simulation-enhanced data inspection and task-specific $\pi_{0.5}$ post-training.

Keywords: Vision-Language-Action Models; Imitation Learning; Robotic Manipulation; Simulation Augmentation; Robot Post-Training.

1. Project Overview

1.1. Goal

Robotic manipulation requires a policy to connect semantic instructions, visual observations, proprioceptive robot states, and continuous motor commands. Generalist vision-language-action (VLA) models such as π_0 and $\pi_{0.5}$ provide a strong pretrained starting point for this problem, but direct deployment on a low-cost robot arm is still limited by embodiment mismatch, camera layout, workspace geometry, and the visual appearance of the target objects [1–3]. Our goal is to fine-tune $\pi_{0.5}$ for two tabletop manipulation tasks on the SO101 6-DOF robot arm:

- Task 1: pick a white milk box.
- Task 2: pull a tissue out of the tissue pack.

For each task, we collect 50 teleoperated demonstrations, convert the data into the LeRobot/OpenPI training format, replay trajectories in Isaac Sim, and post-train the action model with AdamW. The final robot policy takes front-view images, wrist/hand-camera images, robot states, and a task instruction as input, and outputs a 6-dimensional continuous action for the SO101 arm.

1.2. System Pipeline

Figure 1 summarizes the full workflow. Demonstrations are first collected through leader-follower teleoperation. The raw demonstrations are cleaned to remove robot stuttering and incorrect actions, then formatted as LeRobot datasets containing robot states, camera images, and language task instructions. A trajectory augmentation stage replays collected trajectories in Isaac Sim and varies material, lighting, and surface appearance to increase observation diversity. The final stage performs OpenPI post-training and evaluates the model with loss, training speed, task success, completion time, and error analysis.

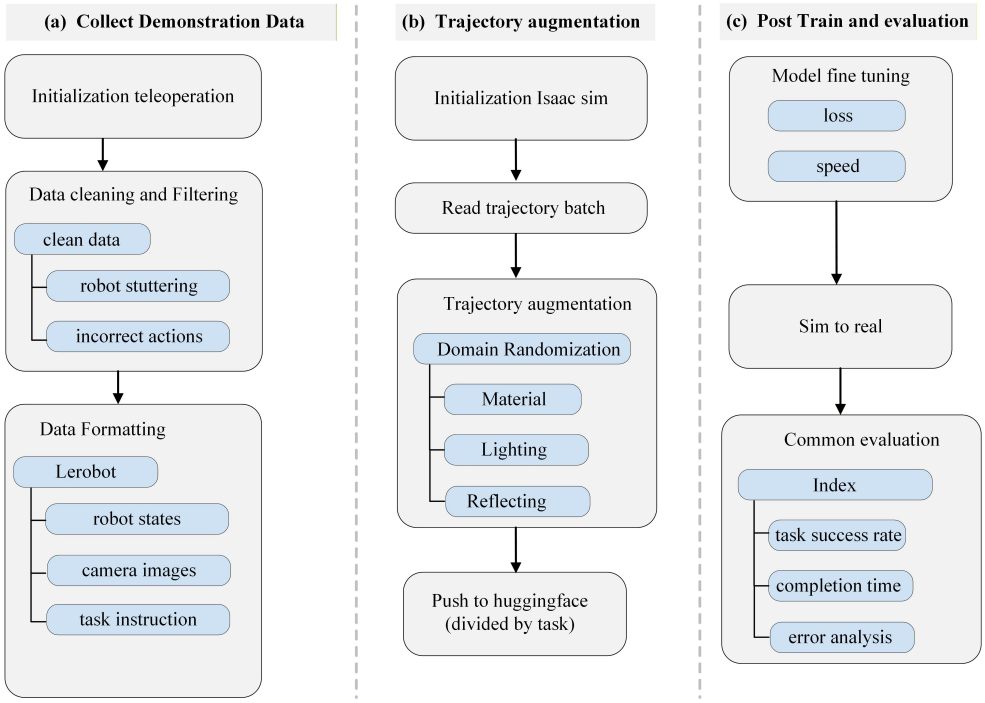


Figure 1: End-to-end project pipeline from teleoperated demonstration collection to trajectory augmentation, post-training, and evaluation.

2. Related Work

2.1. Vision-Language-Action Models

Recent robot learning systems increasingly learn policies that map images and language directly to robot actions. RT-1 demonstrated that transformer policies trained on large-scale robot demonstrations can generalize across many real-world manipulation tasks [4]. RT-2 further connected web-scale vision-language pretraining with robotic control by representing robot actions in a tokenized form [5]. The π_0 model introduced a flow-matching VLA policy for generalist robot manipulation, while $\pi_{0.5}$ extends this

family with stronger open-world generalization through heterogeneous co-training and high-level semantic supervision [1, 2]. In this project, $\pi_{0.5}$ is used as the foundation policy and is adapted to the SO101 arm through task-specific post-training.

2.2. Teleoperation and Robot Datasets

High-quality demonstrations remain important for aligning a generalist policy with a specific robot embodiment. Leader-follower teleoperation records human-guided motions while preserving the robot's joint limits, gripper behavior, and visual perspective. LeRobot provides a practical open-source stack for robot interfaces, dataset formatting, policy training, and evaluation [6]. Our data pipeline follows this idea: demonstrations are stored with synchronized front-camera images, hand-camera images, robot states, actions, and language instructions, which makes the dataset directly usable by OpenPI post-training.

2.3. Simulation Augmentation and Sim-to-Real

Simulation can increase visual and physical diversity when real data collection is expensive. Isaac Sim provides a physically based robotics simulation environment and Replicator tools for synthetic data generation and domain randomization [7, 8]. Domain randomization has been widely used to narrow the sim-to-real gap by varying appearance, lighting, poses, textures, and other scene parameters during training [9]. RoboWheel also highlights a related direction: reconstructing and retargeting manipulation trajectories, then using Isaac Sim augmentation to enrich robot learning data across embodiments [10]. Inspired by these methods, our simulation stage focuses on replaying already collected SO101 trajectories and varying the visual domain while preserving the original spatial motion.

3. Design and Implementation

3.1. Teleoperation Platform

The hardware platform consists of a SO101 leader arm, a SO101 follower arm, and monocular cameras for observation. The leader arm is operated by a human demonstrator, while the follower arm executes the corresponding motion and records synchronized states and actions. This design avoids manual action labeling and ensures that demonstrations remain kinematically feasible for the follower robot.

The SO101 arm state is represented by 6 robot-state dimensions, and each action is a 6-dimensional continuous command. The joint configuration contains shoulder pan, shoulder lift, elbow flex, wrist flex, wrist roll, and gripper control. The dual-view visual setup uses a front camera for global workspace context and a hand camera for close-range gripper-object interaction. During data cleaning, we remove segments with visible stuttering, interrupted grasps, or incorrect action execution before converting the episodes into the training dataset.

Figure 2: Teleoperation platform setup with the leader arm, follower arm, and monocular camera.

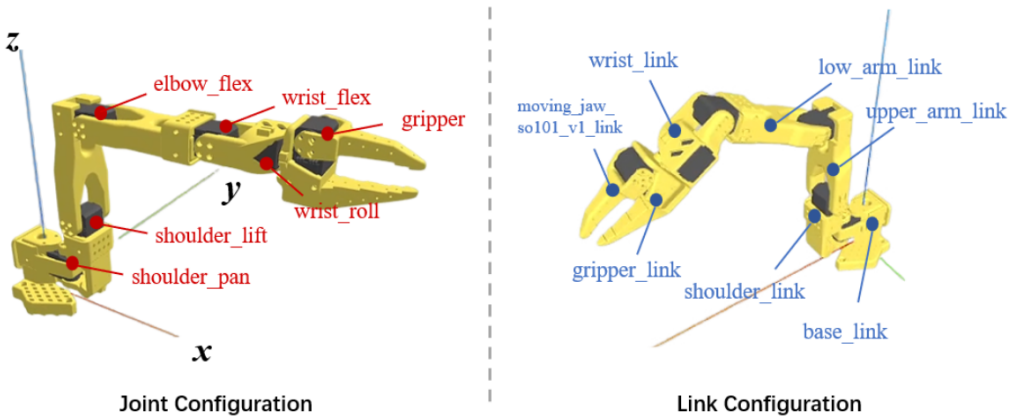
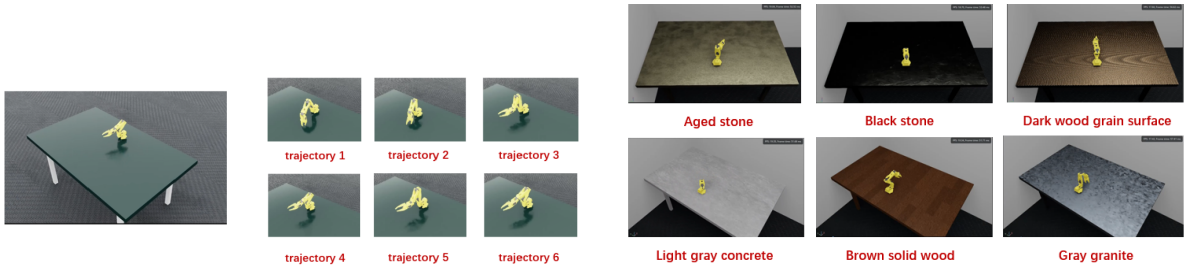


Figure 3: SO101 6-DOF robot arm joint and link configuration.

3.2. Trajectory Augmentation in Isaac Sim

The simulation module is designed as an Isaac Sim 5.1 trajectory replay and rendering pipeline. Instead of inventing new robot motions, the simulator replays collected SO101 demonstrations and changes the visual domain during rendering. This keeps the action sequence anchored to real teleoperation while exposing the model to more varied appearances.

The augmentation parameters include tabletop material, lighting, reflection, and rendered surface texture. The examples in Figure 4 include aged stone, black stone, dark wood grain, light gray concrete, brown solid wood, and gray granite surfaces. This design targets appearance robustness: the model should learn the motion structure and object interaction instead of overfitting to one tabletop texture.



(a) Replay of collected trajectories.

(b) Domain-randomized rendering.

Figure 4: Isaac Sim trajectory replay and augmentation. The augmentation stage randomizes table materials and surface appearances while preserving the trajectory geometry.

4. Experiments and Results

4.1. Dataset Collection

We designed two real-world manipulation tasks and collected 50 teleoperated trajectories for each task. Every episode contains a task instruction, front-camera observations, hand-camera observations, 6-dimensional robot state, and 6-dimensional action. Figure 5 shows representative observation samples from both camera views.

Table 1: Collected demonstration datasets.

Task	Instruction	Trajectories	Observations
Task 1	Pick white milk box	50	Front camera, hand camera, robot state
Task 2	Pull tissue from tissue pack	50	Front camera, hand camera, robot state

(a) Task 1: pick white milk box.

(b) Task 2: pull a tissue out of the tissue pack.

Figure 5: Dataset visualization with front-camera sequences in the first row and hand-camera sequences in the second row.

4.2. Post-Training Setup

The model is fine-tuned using the OpenPI post-training framework with AdamW. Images are resized to 224×224, robot state and action dimensions are both 6, the global batch size is 64, and the peak learning rate is 5×10^{-5}. Training was run on 8 NVIDIA A800 GPUs. The planned training schedule contains 4000 total steps, with a measured throughput of about 5.8–6.0 seconds per step and a total runtime of about 7 hours and 34 minutes.

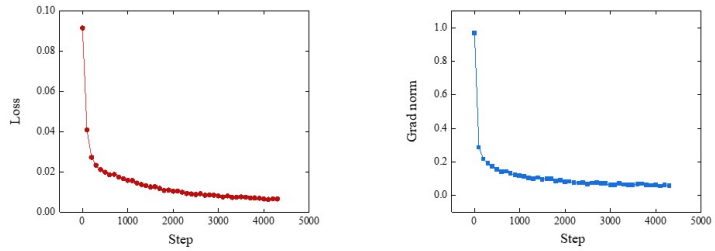
Table 2: Training hyperparameters.

Category	Parameter	Value
Image preprocess	Image size	224×224
State dim	Robot state	6
Action dim	Action dimension	6
Batch size	Global batch	64
Optimizer	Optimizer	AdamW
Learning rate	Peak LR	5×10^{-5}
Hardware	GPUs	$8 \times$ NVIDIA A800
Schedule	Total steps	4000
Speed	Time per step	5.8–6.0 s/step
Runtime	Total time	about 7 h 34 min

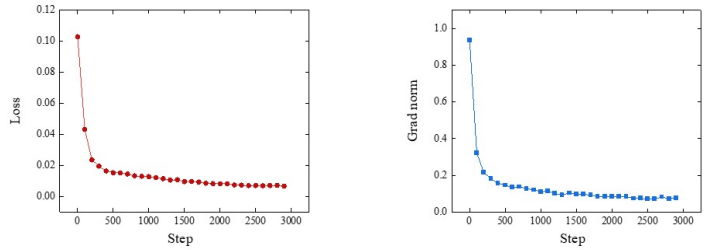
For a batch of N training samples, the action-learning loss is written as $\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \|v_\theta(o_i, s_i, t_i) - v_i^\star\|_2^2$, where o_i is the visual observation, s_i is the robot state, t_i is the task instruction, and $v_i^\star$ is the target action-flow supervision. The gradient norm is monitored as $\|\nabla_\theta \mathcal{L}\|_2$ to check whether optimization remains stable.

4.3. Training Curves

Figure 6 reports the loss and gradient norm logs exported from training. For the milk-box task, the loss decreases from 0.0915 at step 0 to 0.00655 at step 4000, a reduction of about 92.8%. The corresponding gradient norm decreases from 0.966 to 0.0619, a reduction of about 93.6%. For the tissue task, the exported log decreases from 0.1028 to 0.00690 by the final logged step, while the gradient norm decreases from 0.935 to 0.0711. These trends indicate stable convergence under the selected batch size, optimizer, and learning rate.



(a) Task 1 loss. (b) Task 1 gradient norm.



(c) Task 2 loss. (d) Task 2 gradient norm.

Figure 6: Training loss and gradient norm curves for the two manipulation tasks.

5. Conclusion

1. We completed a task-specific $\pi_{0.5}$ post-training pipeline for SO101 6-DOF manipulation, including teleoperation, data cleaning, dataset formatting, Isaac Sim replay, and OpenPI post-training.
2. The final dataset contains 100 real teleoperated demonstrations across two tasks: 50 trajectories for picking a white milk box and 50 trajectories for pulling a tissue out of the tissue pack.
3. The milk-box training loss decreased by about 92.8% by step 4000, and the tissue-task exported log reached a final loss of about 0.00690. The gradient-norm curves for both tasks show stable optimization.
4. Future work should expand repeated physical evaluation, report task success rate and completion-time statistics under different tabletop layouts, and add more object instances to improve robustness.

References

[1] Physical Intelligence, K. Black, N. Brown, D. Driess, et al., π_0: A vision-language-action flow model for general robot control (2024). arXiv:2410.24164, doi:10.48550/arXiv.2410.24164.
URL https://arxiv.org/abs/2410.24164

[2] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, et al., $\pi_{0.5}$: A vision-language-action model with open-world generalization (2025). arXiv:

2504.16054, doi:10.48550/arXiv.2504.16054.
URL https://arxiv.org/abs/2504.16054

[3] Physical Intelligence, openpi: Open-source models and packages for robotics, https://github.com/Physical-Intelligence/openpi (2025).

[4] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, et al., Rt-1: Robotics transformer for real-world control at scale, Robotics: Science and Systems (2023).
URL https://arxiv.org/abs/2212.06817

[5] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, D. Driess, P. Florence, C. Finn, B. Ichter, et al., Rt-2: Vision-language-action models transfer web knowledge to robotic control (2023). arXiv:2307.15818, doi:10.48550/arXiv.2307.15818.
URL https://arxiv.org/abs/2307.15818

[6] Hugging Face, Lerobot: State-of-the-art machine learning for real-world robotics, https://github.com/huggingface/lerobot (2024).

[7] NVIDIA, Nvidia isaac sim: Robotics simulation and synthetic data generation, https://developer.nvidia.com/isaac/sim (2026).

[8] NVIDIA, Isaac sim replicator: Perception data generation, https://docs.isaacsim.omniverse.nvidia.com/5.0.0/replicator_tutorials/index.html (2026).

[9] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, Domain randomization for transferring deep neural networks from simulation to the real world, in: IEEE/RSJ International Conference on Intelligent Robots and Systems, 2017, pp. 23–30. doi:10.1109/IROS.2017.8202133.
URL https://arxiv.org/abs/1703.06907

[10] Y. Zhang, Z. Gao, S. Li, L.-H. Chen, K. Liu, R. Cheng, X. Lin, J. Liu, Z. Li, J. Feng, Z. He, J. Lin, Z. Huang, Z. Liu, H. Wang, Robowheel: A data engine from real-world human demonstrations for cross-embodiment robotic learning (2025). arXiv:2512.02729, doi:10.48550/arXiv.2512.02729.
URL https://arxiv.org/abs/2512.02729